

Linear Regression Model for Diabetes Progression

Assignment 1 Part 1 - Regression and Regularisation

Somaan Salim Mirza

ECS708P - Machine Learning

November 2025

Student ID: 200356086

Q5: Weight Interpretation and Patient Predictions

The model's learned weights reveal the relative importance of each feature in predicting diabetes progression (see Appendix A, Figure 1). BMI (26.36) emerged as the strongest positive predictor, consistent with established medical research linking obesity to diabetes complications. The model assigned a negative weight to SEX (-11.44), indicating that when controlling for all other clinical factors, males tend to have higher disease progression despite females showing slightly higher average progression in the raw data (155.67 vs 149.82, see Appendix A, Figure 2). This demonstrates the importance of multivariate regression over univariate analysis.

Two hypothetical patients were created to test the model (see Appendix A, Figure 3). Patient 1 (25-year-old female, BMI 18) received a predicted progression of 42.61, while Patient 2 (50-year-old male, BMI 28) scored 227.72. The higher prediction for Patient 2 aligns with expectations given the combination of higher BMI, male sex, and contributions from other clinical markers (S5: 20.79, BP: 16.61). The model successfully integrates multiple features to produce clinically meaningful predictions.

Q6: Effects of Learning Rate

Different learning rates were tested to evaluate their impact on convergence and performance:

Learning Rate (α)	**Train Error**	**Test Error**	**Status**
0.00001	28968.54	26918.10	Diverged
0.0001	25427.83	23577.98	Diverged
0.01	2892.74	2865.06	Good
0.1	2894.29	2865.96	Good

Very low learning rates (0.00001, 0.0001) failed to converge within 100 iterations, with the model making insufficient progress toward the minimum. The optimal learning rates (0.01, 0.1) achieved low training and test errors with <1% difference (28 units), indicating good generalisation without overfitting. The $\alpha = 0.1$ rate offers faster convergence while maintaining comparable performance to $\alpha = 0.01$, making it the optimal choice for this dataset. Cost convergence behaviour is visualised in Appendix B, Figure 4.

Q8: Parameter Optimisation

Using the optimal learning rate ($\alpha = 0.1$), different regularisation strengths were tested (see Appendix C, Figures 5-8). Without regularisation ($\lambda = 0$), the fifth-order polynomial fits the training data closely using its full flexibility. Light regularisation ($\lambda = 0.1$) produces a smooth curve that balances fit quality with weight constraints, demonstrating optimal performance. Moderate regularisation ($\lambda = 1.0$) further smooths the hypothesis while maintaining reasonable fit. Heavy regularisation ($\lambda = 5.0$) produces an overly constrained, nearly linear curve that underfits the data.

The experiments demonstrate the critical trade-off: too little regularisation allows excessive flexibility, while too much causes underfitting. The optimal range ($\lambda \approx 0.1-1.0$) balances model complexity with fit quality, producing effective polynomial regression without overfitting.

A Appendix A (Q5): Weight Analysis and Patient Predictions

```
features_list = ['AGE', 'SEX', ' BMI', 'BP', 'S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'BIAS']

print("="*70)
print("Q5 Results:")
print("="*70)
print(f"{'Feature':<8s} {'Weight':>10s}")
print("-"*25)
for name, weight in zip(features_list, model.weight[0]):
  print(f"{name:<8s} {weight.item():>10.2f}")
```

Figure 1: Learned feature weights showing BMI as strongest predictor

```
#sex checking
print("Average dissease progression by sex")
print(diabetes_db.groupby('SEX')['Y'].mean())

Average dissease progression by sex
SEX
1    149.021277
2    155.666667
Name: Y, dtype: float64
```

Figure 2: Average disease progression by sex

```
#Features for diabetes analysis
bmi_weight = model.weight[0,2].item()
sex_weight = model.weight[0,1].item()

#p1
p1 = torch.tensor([[25, 2, 18, 79, 130, 64.8, 61, 2, 4.1897, 68]]).float()
p1_norm = (p1 - mu) / sigma
p1_norm = torch.cat([p1_norm, torch.ones(1, 1)], dim=1)
pred1 = model(p1_norm).item()

#p2
p2 = torch.tensor([[50, 1, 28, 103, 229, 162.2, 60, 4.5, 6.107, 124]]).float()
p2_norm = (p2 - mu) / sigma
p2_norm = torch.cat([p2_norm, torch.ones(1, 1)], dim=1)
pred2 = model(p2_norm).item()

print(f"\nPatient 1:  {pred1:.2f}")
print(f"\nPatient 2:  {pred2:.2f}")
```

Figure 3: Patient predictions

B Appendix B (Q6): Learning Rate Analysis

```
learning_rates = [0.00001, 0.0001, 0.01, 0.1]
results = []

for alpha in learning_rates:
  lr_model = LinearRegression(x_train.shape[1])
  cost_hist = []

  #Train for x interations
  for i in range(100):
    pred = lr_model(x_train)
    cost = mean_squared_error(y_train, pred)
    cost_hist.append(cost.item())
    gradient_descent_step(lr_model, x_train, y_train, pred, alpha)

  #Evaluation
  train_error = mean_squared_error(y_train, lr_model(x_train))
  test_error = mean_squared_error(y_test, lr_model(x_test))

  #Status descriptor
  if train_error < 3000 and test_error < 3000:
    status = "Good"
  elif train_error > 10000 or test_error > 10000:
    status = "Diverged"
  else:
    status = "slow"

  print(f"Alpha = {alpha}|Train: {train_error}|Test: {test_error}|Status {status}")
```

Figure 4: Cost convergence comparison for different learning rates (α)

C Appendix C (Q8): Parameter Optimisation

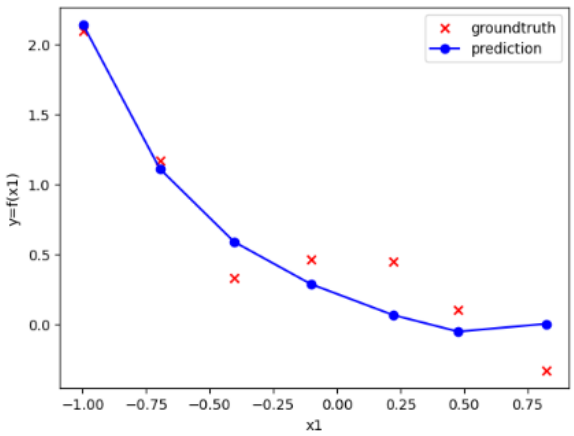


Figure 5: $\lambda = 0$: No regularisation

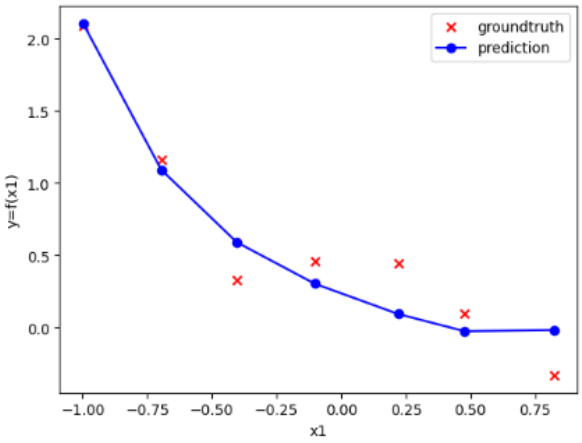


Figure 6: $\lambda = 0.1$: Light regularisation

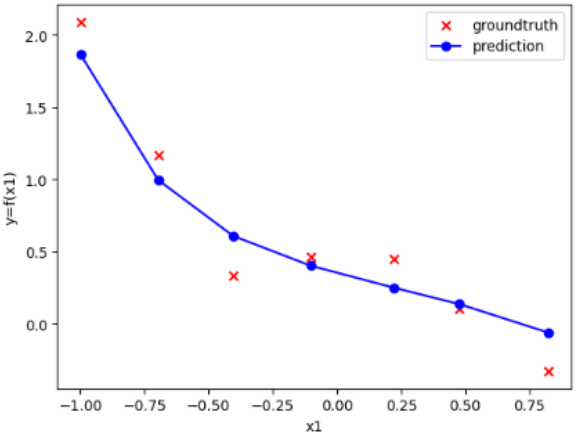


Figure 7: $\lambda = 1.0$: Moderate regularisation

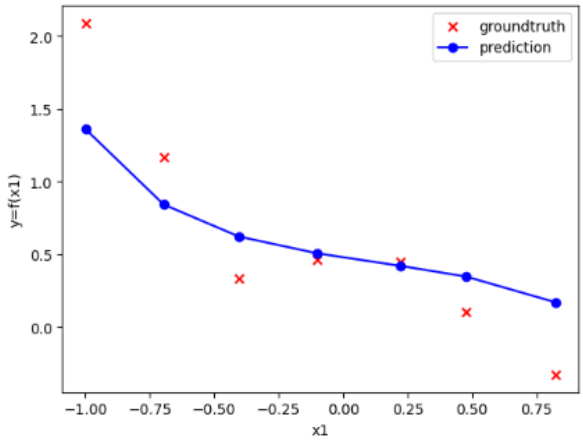


Figure 8: $\lambda = 5.0$: Heavy regularisation



Logistic Regression for Iris Classification

Assignment 1 Part 2 - Classification

Somaan Salim Mirza

ECS708P - Machine Learning

November 2025

Student ID: 200356086

Q1: Feature Normalisation

The raw training data shows features with a large scale range, with sepal length ranging 4.5-8cm and sepal width 2-4.5cm (see Appendix A, Figures 1 and 2). After normalisation, all features become centered at mean ≈ 0 with unit variance (std ≈ 1). The normalised data allows the model to train features on comparable scales, centered around the origin. This standardisation also prevents features with larger magnitudes from dominating gradient descent updates.

Q5: Decision Boundary Analysis

The learned decision boundary successfully separates the setosa class from non-setosa classes in the test set (see Appendix B, Figures 3 and 4). The linear boundary cleanly divides red points (setosa, upper-left) from blue points (non-setosa, lower-right). This matches expectations from the initial visualisation, where setosa was linearly separable from versicolor and virginica. The clear separation demonstrates logistic regression is effective for this binary classification task.

Q7: Multiclass Predictions

Using three trained binary classifiers (setosa, versicolor, virginica) in a 1-vs-all approach, predictions were generated for the test set (see Appendix C, Figures 5 and 6). Several interesting patterns emerge. At first glance, probabilities from the three independent classifiers do not sum to 1, as each operates independently rather than as a unified multiclass model. Second, some samples show ambiguous classifications with similar probabilities across multiple classes, making confident predictions difficult. Third, setosa appears well-separated, with most non-setosa samples showing very low setosa probabilities (close to 0), while certain samples show high setosa confidence. These observations reveal limitations of the 1-vs-all approach, where independent binary classifiers can produce conflicting predictions.

Q8: Classification Accuracy

Applying softmax to normalise predictions and selecting the class with the highest probability, we identify the classifier has achieved 70% accuracy on the test set (21 out of 30 correct predictions) (see Appendix D, Figure 7). This accuracy demonstrates successful learning, particularly for the setosa class which remains accurately classified due to linear separation. However, some confusion persists between versicolor and virginica classes, which overlap significantly in the feature space. This demonstrates that while the 1-vs-all logistic regression approach can achieve solid performance, it still faces challenges with overlapping classes that are not perfectly linearly separable.

Q9: The XOR Problem

Logistic regression cannot solve the XOR problem because it learns only linear decision boundaries. The XOR pattern places same-class points diagonally opposite (red at (0,0)

and (1,1); blue at (0,1) and (1,0)), making linear separation impossible. No single straight line can separate these classes. This represents a fundamental limitation of logistic regression: it cannot model non-linear decision boundaries. More complex models such as neural networks with hidden layers are needed for problems like XOR.

1 Appendix A (Q1): Feature Normalisation

```python
def norm_set(x: torch.Tensor, mu: torch.Tensor, sigma: torch.Tensor) -> torch.tensor:
  #normalisation crucial as features have different scales
  z = (x - mu) / sigma
  return z

x_train_raw = x_train.clone()
x_test_raw = x_train.clone()

mu = x_train.mean(dim=0)
sigma = x_train.std(dim=0)

x_train = norm_set(x_train, mu, sigma)
x_test = norm_set(x_test, mu, sigma)

#verification
print("Q1 Results:")
print(f"  Mean of feature: {x_train.mean(dim=0)[:5]}")
print(f"  Std of feature: {x_train.std(dim=0)[:5]}")

#raw vs normalised training sets
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

#raw
axes[0].scatter(x_train_raw[:, 0], x_train_raw[:, 1], alpha=0.3)
axes[0].set_title('Raw Training Set')
axes[0].set_xlabel('Sepal Length (cm)')
axes[0].set_ylabel('Sepal Width (cm)')
axes[0].grid(alpha=0.3)

#normalised
axes[1].scatter(x_train[:, 0], x_train[:, 1], alpha=0.3)
axes[1].set_title('Normalised Training Set')
axes[1].set_xlabel('Sepal Length (cm)')
axes[1].set_ylabel('Sepal Width (cm)')
axes[1].grid(alpha=0.3)
```

Figure 1: Q1 implementation code

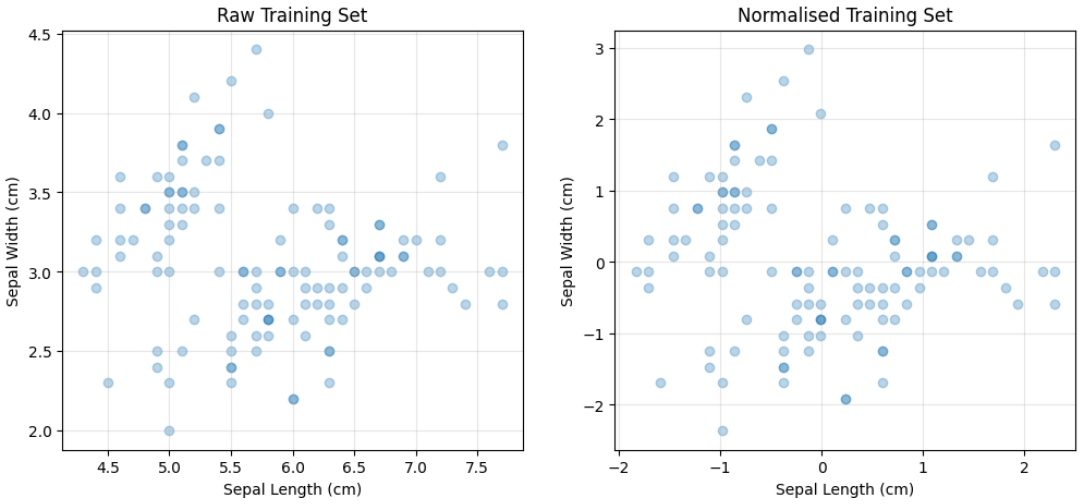


Figure 2: Raw vs normalised training data showing scale standardisation

2 Appendix B (Q5): Decision Boundary

```python
#predictions of test set (1)
test_pred = model(test_set_1)
test_pred_class = (test_pred > 0.5).int()

#plot test sets (scatter plot)
plt.figure(figsize=(8, 6))
plt.scatter(test_set_1[:, 0], test_set_1[:, 1], c=setosa_test.squeeze(), cmap='bwr', alpha=0.3)
plt.xlabel("Sepel Length (normalised)")
plt.ylabel("Sepel Width (normalised)")
plt.title("Decision boundary")

w = model.weight.squeeze()
x1_range = torch.linspace(test_set_1[:, 0].min(), test_set_1[:, 0].max(), 100)
x2_boundary = -(w[0] * x1_range + w[2]) / w[1]
plt.plot(x1_range, x2_boundary, color="black", label="Decision Boundary")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Figure 3: Implementation code

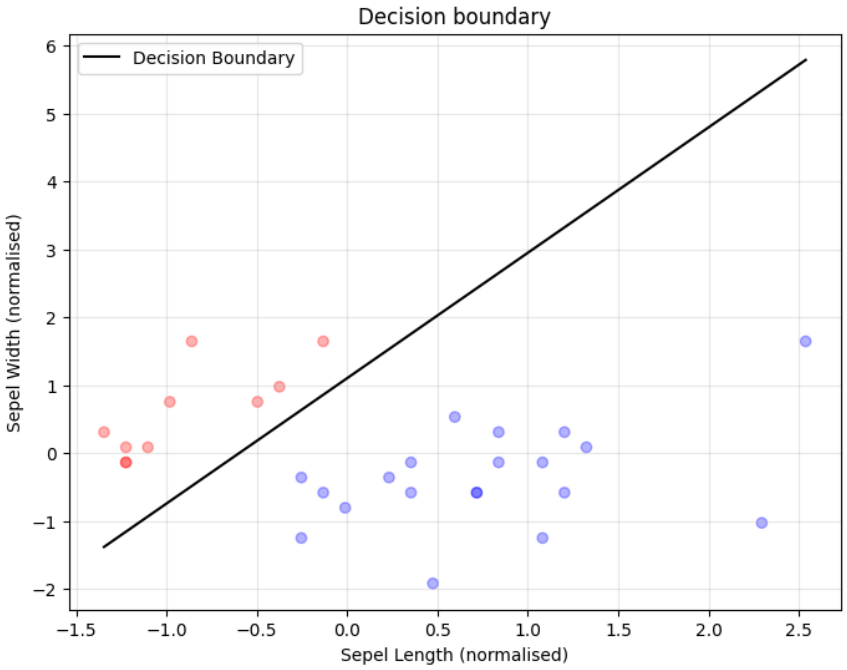


Figure 4: Decision boundary separating setosa from non-setosa classes

3 Appendix C (Q7): Multiclass Predictions

```python
#get predictions from each classifier
setosa_pred = setosa_model(x_test)
veriscolor_pred = veriscolor_model(x_test)
virginica_pred = virginica_model(x_test)

#combined predictions
predictions = torch.cat([setosa_pred, veriscolor_pred, virginica_pred], dim=1)

df = pd.DataFrame(
    predictions.detach().numpy(),
    columns=['setosa', 'veriscolor', 'virginica']
)

print("Table of predictions:")
df.head(10)
```

Figure 5: Implementation code

Table of predictions:

	setosa	veriscolor	virginica
0	0.030168	0.541836	0.527355
1	0.999957	0.467023	0.324662
2	0.000003	0.801264	0.681047
3	0.038258	0.550617	0.554383
4	0.006551	0.620940	0.560129
5	0.999659	0.350603	0.378708
6	0.297777	0.442717	0.533114
7	0.002191	0.731467	0.634482
8	0.000411	0.426077	0.637594
9	0.074182	0.437551	0.543702

Figure 6: Predictions from three binary classifiers (first 10 samples)

4 Appendix (Q8): Classification Accuracy

```python
#softmax conversion
predictions_s = F.softmax(predictions, dim=1)

#predicted class
predicted_class = predictions_s.argmax(dim=1)

#acquire ground truth
true_class = y_test.argmax(dim=1)

#calculate accuracy
correctness = (predicted_class == true_class).sum().item()
total = y_test.shape[0]
accuracy = correctness / total

print("Accuracy Calculations:")
print('='*70)
print(f"Correct Predictions: {correctness}")
print(f"Total Predictions: {total}")
print(f"Accuracy (%): {accuracy * 100}")
```

```
Accuracy Calculations:
======================================================================
Correct Predictions: 21
Total Predictions: 30
Accuracy (%): 70.0
```

Figure 7: Test set accuracy after softmax normalisation



Neural Networks for Pattern Recognition

Assignment 1 Part 3 - Neural Networks

Somaan Salim Mirza

ECS708P - Machine Learning

November 2025

Student ID: 200356086

Question 1: Neural Network Implementation

The forward and backward pass methods were implemented for a neural network with one hidden layer (see Appendix A, Figures 1 and 2). The forward pass computes hidden layer outputs using sigmoid activation, then produces final outputs through the output layer. The backward pass implements backpropagation by computing output gradients, propagating errors to hidden layers, and updating weights using $w = w - \text{lr} \times \text{gradient} \times \text{input}$. Testing confirms weights update correctly from initial zeros to learned values (see Figures 3 and 4).

Random weight initialisation is crucial for neural networks rather than initialising all weights to zero. If all weights start at zero, all neurons in the same layer compute identical outputs during forward propagation. During backpropagation, they receive identical gradients and update identically, remaining symmetric throughout training. This symmetry breaking problem means the network cannot learn different features. All hidden neurons essentially become redundant copies. Random initialisation breaks this symmetry, allowing each neuron to learn different patterns and features from the data, which is essential for the network's representational capacity.

Question 2: XOR Problem

A neural network with 2 hidden neurons was trained on the XOR problem over 5000 iterations with learning rate 0.5 (see Appendix B, Figures 5 and 6). BCE loss decreased from 0.72 to 0.52, showing successful learning. Final predictions [0.33, 0.95, 0.31, 0.32] correctly classify 3 out of 4 XOR cases compared to true labels [0, 1, 1, 0].

The neural network solves XOR by learning a non-linear transformation through its hidden layer. The XOR pattern cannot be separated by a single straight line, as points of the same class are positioned diagonally opposite (class 0 at (0,0) and (1,1), class 1 at (0,1) and (1,0)). The hidden layer transforms the input space into a new representation where the classes become linearly separable. Each hidden neuron learns a different linear boundary, and their combined outputs create a representation that the output layer can then separate with a single linear decision boundary. This two-stage process, hidden layer transformation followed by linear separation, enables neural networks to solve problems that are not linearly separable in the original input space.

Question 3: MLP Performance on Iris Dataset

MLPs with varying hidden layer sizes (1, 2, 4, 8, 16, 32 neurons) were trained on the Iris dataset for 500 epochs (see Appendix C, Figures 7 through 11). Performance improves dramatically with capacity. With 1 neuron, accuracy is only 33%, insufficient for the task. At 4 neurons, performance reaches 80% train and 86.7% test. The network continues improving through 8 neurons (85%, 86.7%) and 16 neurons (90.8%, 90%). Beyond 16 neurons, performance plateaus at 91.7% train and 93.3% test, showing diminishing returns. This pattern suggests 8-16 hidden neurons provide optimal capacity for the Iris dataset's complexity.

Test accuracy equals or exceeds training accuracy across most configurations, indicating strong generalisation without overfitting. The network effectively learns decision boundaries without memorising training data. Compared to logistic regression from Lab

4 (70% accuracy), even an 8-neuron MLP (86.7% test) substantially outperforms the linear classifier. This improvement demonstrates neural networks' ability to learn non-linear decision boundaries, addressing the class overlap limitations between versicolor and virginica observed in the logistic regression model.

1 Appendix A (Q1): Neural Network Implementation

```python
class NeuralNetwork(nn.Module):
  def __init__(self, num_inputs, num_neurons, num_outputs):
    super().__init__()
    self.hidden_layer = nn.ModuleList([LogisticRegression(num_inputs) for i in range(num_neurons)])
    self.output_layer = nn.ModuleList([LogisticRegression(num_neurons+1) for i in range(num_outputs)]) # +1 for the bias

  def forward(self, x: torch.tensor) -> torch.tensor:
    y_hidden = torch.ones(len(self.hidden_layer) + 1) # +1 for the bias

    #compute hidden layer outputs
    for i in range(len(self.hidden_layer)):
      y_hidden[i] = self.hidden_layer[i](x)

    #compute output layer outputs
    y = torch.zeros(len(self.output_layer))
    for i in range(len(self.output_layer)):
      y[i] = self.output_layer[i](y_hidden)
    return y

  def backward_pass(self, x: torch.tensor, y: torch.tensor, lr: float) -> None:
    #Forward pass to get all activations
    y_hidden = torch.ones(len(self.hidden_layer) + 1) # +1 for the bias
    for i in range(len(self.hidden_layer)):
      y_hidden[i] = self.hidden_layer[i](x)

    y_pred = torch.zeros(len(self.output_layer))
    for i in range(len(self.output_layer)):
      y_pred[i] = self.output_layer[i](y_hidden)
```

Figure 1: Neural network class with forward and backward pass implementation (part 1)

```python
#get output layer gradients and update weights accorrindly
errors = []
for i in range(len(self.output_layer)):
  error = y_pred[i] - y[i]

#gradient - sigmoid deriv * error
  grad = y_pred[i] * (1-y_pred[i])* error

#update the weights - w - lr * gradient * input???
self.output_layer[i].weight.data -= lr * grad * y_hidden.unsqueeze(0) #ensures shape matches for multiplciation
errors.append(error*y_pred[i] * (1-y_pred[i]))

#compute hidden layer gradients
for i in range(len(self.hidden_layer)):
  back_error = 0
  for j in range(len(self.output_layer)):
    back_error += errors[j]*self.output_layer[j].weight.data[0,i]

#new gradient with backpropagated error
  new_grad = y_hidden[i] * (1-y_hidden[i]) * back_error

#update weights
  self.hidden_layer[i].weight.data -= lr * new_grad * x

return
```

Figure 2: Neural network class with forward and backward pass implementation (part 2)

```
print("Before:")
print(model.hidden_layer[0].weight)

Before:
Parameter containing:
tensor([[0., 0., 0.]])
```

Figure 3: Weights before training (all zeros)

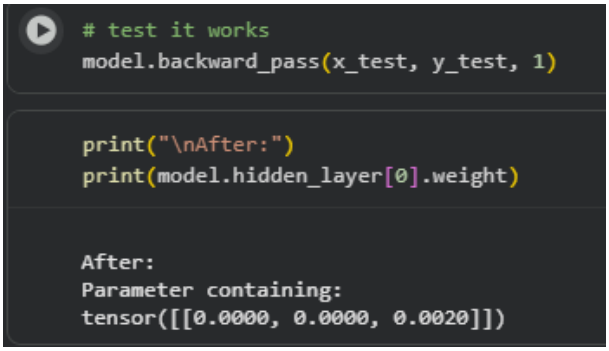


Figure 4: Weights after one backward pass showing successful updates

2 Appendix B (Q2): - XOR Problem

```python
def bce(y_true: torch.Tensor, y_pred: torch.Tensor) -> torch.Tensor:
  cost = -(y_true * (y_pred).log() + (1- y_true) * (1- y_pred).log())
  return cost.mean()
```

```python
x1 = torch.tensor([0., 0., 1., 1.])
x2 = torch.tensor([0., 1., 0., 1.])
bias = torch.ones_like(x1)
x = torch.stack([x1, x2, bias]).permute(1, 0)
print(x.shape)
y = torch.tensor([0., 1., 1., 0.])

lr = 0.5
iterations = 5000
model = NeuralNetwork(num_inputs=3, num_neurons=2, num_outputs=1)

errors = list()
for ep in range(iterations):
    error = 0
    for i in range(x.shape[0]):
        y_pred = model(x[i])
        error += bce(y[i:i+1], y_pred)  #use slicing to preserve dimension
        model.backward_pass(x[i], y[i:i+1], lr)
    errors.append(error/x.shape[0])

plt.plot(list(range(iterations)), errors)
plt.xlabel('Iteration')
plt.ylabel('BCE Loss')
plt.title('XOR Training Loss')
plt.show()

#test predictions
y_pred = torch.zeros_like(y)
for i in range(x.shape[0]):
    y_pred[i] = model(x[i])

print("True labels:", y)
print("Predictions:", y_pred)
```

Figure 5: XOR training code with BCE loss function

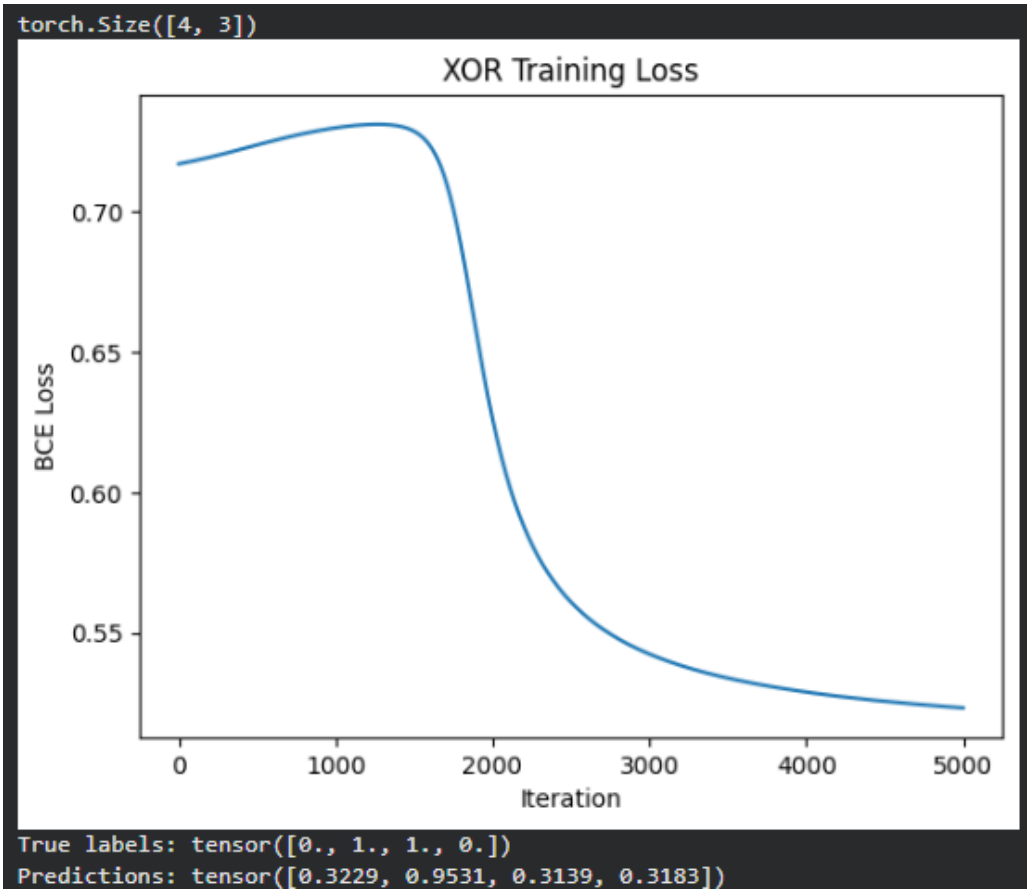


Figure 6: XOR training loss over 5000 iterations (lr=0.5)

3 Appendix C (Q3): MLP Performance Analysis

```python
#mlp class
class MLP(nn.Module):
  def __init__(self, d_input, d_output, d_hidden):
    super().__init__()
    self.fc1 = nn.Linear(d_input, d_hidden)
    self.fc2 = nn.Linear(d_hidden, d_output)

  def forward(self, x):
    h = torch.relu(self.fc1(x))
    out = self.fc2(h)
    return out

#train/evaluate
def model_eval(hidden_neurons, epochs=500, lr=0.01):
  model = MLP(d_input=4, d_hidden=hidden_neurons, d_output=3)
  loss_fn = nn.CrossEntropyLoss()
  optimiser = optim.SGD(model.parameters(), lr=lr)

  train_acc_history = []
  test_acc_history = []

  for epoch in range(epochs):
    #training begins here
    model.train()
    pred_train = model(x_train)
    loss = loss_fn(pred_train, y_train)

    optimiser.zero_grad() #clears old gradients
    loss.backward() #computes new ones
    optimiser.step() #updates weights accordingly

    #evaluation
    if epoch % 10 == 0:
      model.eval()
      with torch.no_grad():
        pred_train = model(x_train)
        train_correctness = (pred_train.argmax(1) == y_train).sum().item()
        train_acc = train_correctness/len(y_train)
```

Figure 7: MLP class implementation

```
        #test accuracy
        pred_test = model(x_test)
        test_correctness = (pred_test.argmax(1) == y_test).sum().item()
        test_acc = test_correctness/len(y_test)

        train_acc_history.append(train_acc)
        test_acc_history.append(test_acc)

  return train_acc_history[-1], test_acc_history[-1], train_acc_history, test_acc_history
```

```
#experiment run
configs = [1,2,4,8,16,32]
train_results = []
test_results = []

print("training networks with different hidden neuron architecutres")
for h in configs:
  train_acc, test_acc, train_acc_history, test_acc_history = model_eval(h, epochs=500)
  train_results.append(train_acc)
  test_results.append(test_acc)
  print(f"Hidden neurons: {h}")
  print(f"Train accuracy: {train_acc}")
  print(f"Test accuracy: {test_acc}")

#plot
plt.figure(figsize=(8,6))
plt.plot(configs, train_results, label='Train Accuracy')
plt.plot(configs, test_results, label='Test Accuracy')
plt.xlabel('Number of Hidden Neurons')
plt.ylabel('Accuracy')
plt.title('Accuracy vs. Number of Hidden Neurons')
plt.legend()
plt.show()
```

Figure 8: Model evaluation function

```
        #test accuracy
        pred_test = model(x_test)
        test_correctness = (pred_test.argmax(1) == y_test).sum().item()
        test_acc = test_correctness/len(y_test)

        train_acc_history.append(train_acc)
        test_acc_history.append(test_acc)

  return train_acc_history[-1], test_acc_history[-1], train_acc_history, test_acc_history
```

```
#experiment run
configs = [1,2,4,8,16,32]
train_results = []
test_results = []

print("training networks with different hidden neuron architecutres")
for h in configs:
  train_acc, test_acc, train_acc_history, test_acc_history = model_eval(h, epochs=500)
  train_results.append(train_acc)
  test_results.append(test_acc)
  print(f"Hidden neurons: {h}")
  print(f"Train accuracy: {train_acc}")
  print(f"Test accuracy: {test_acc}")

#plot
plt.figure(figsize=(8,6))
plt.plot(configs, train_results, label='Train accuracy')
plt.plot(configs, test_results, label='Test accuracy')
plt.xlabel('Number of hidden neurons')
plt.ylabel('Accuracy')
plt.title('Accuracy vs. Number of hidden neurons')
plt.legend()
plt.show()
```

Figure 9: Experiment code

```
training networks with different hidden neuron architecutres
Hidden neurons: 1
Train accuracy: 0.3333333333333333
Test accuracy: 0.3333333333333333
Hidden neurons: 2
Train accuracy: 0.6583333333333333
Test accuracy: 0.6
Hidden neurons: 4
Train accuracy: 0.8
Test accuracy: 0.8666666666666667
Hidden neurons: 8
Train accuracy: 0.85
Test accuracy: 0.8666666666666667
Hidden neurons: 16
Train accuracy: 0.9083333333333333
Test accuracy: 0.9
Hidden neurons: 32
Train accuracy: 0.9166666666666666
Test accuracy: 0.9333333333333333
```

Figure 10: Numerical results for all network configurations

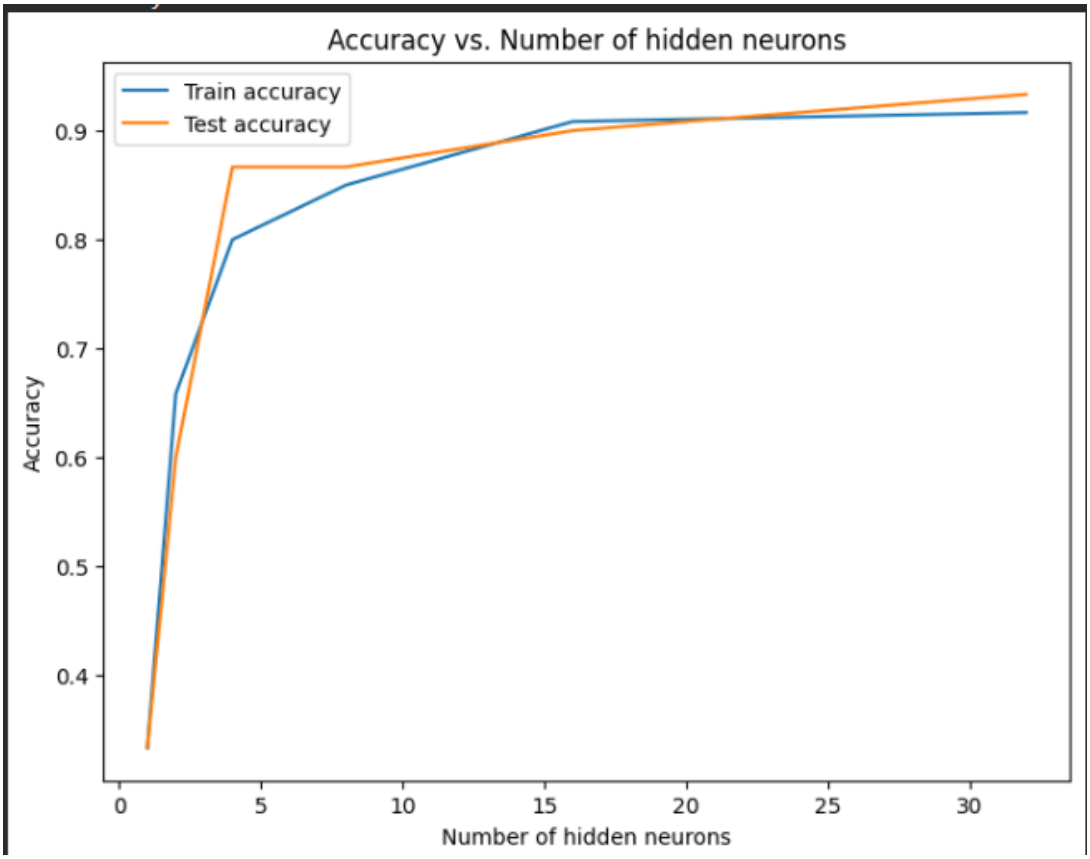


Figure 11: Training and test accuracy vs hidden layer size



Unsupervised Learning

Assignment 2 - Density Estimation

Somaan Salim Mirza

ECS708P - Machine Learning

November 2025

Student ID: 200356086

Q1: Vowel Distribution (F1 vs F2)

Figure 1 (see Appendix A) shows F1 versus F2 formant frequencies across the 10 vowel classes in the Peterson & Barney dataset. The plot displays distinct cluster patterns across the formant space, where each vowel occupies a region corresponding to a distinct articulatory configuration.

Using Peterson and Barney's paper, the dataset contains recordings from 76 speakers (33 men, 28 women, 15 children) producing 10 monosyllabic words: heed, hid, head, had, hod, hawed, hood, who'd, hud, and heard. These correspond to the 10 vowel phonemes: IY, IH, EH, AE, AA, AO, UH, UW, AH, and ER. (Peterson & Barney, 1952). The vowel space represent resonance in the vocal tract, where F1 relates to tongue height, and F2 relates to tongue position.

From Figure 1, it is clear that vowels such as IY (orange), UW (blue), and ER (green) are clearly separated in the F1-F2 plane. (Klautau, 2002). But, this does vary considerably, as other vowel pairs demonstrate overlap that may complicate classification tasks further down the pipeline. For example, AA and AO show clear overlap with multiple other pairs. The degree of overlap observed in Figure 1 and from Klautau suggests that F1 and F2 alone are insufficient for reliable classification of all vowel pairs.

Q2: MoG Training (K=3, K=6)

Figures 2 & 3 (See Appendix B) show two separate runs of the Mixture of Gaussian models, using k=3 components for phoneme class 1. The EM algorithm ran 100 iterations in each case, showing successful convergence in both instances.

The main observation made is that the two runs produce different configurations of the Gaussian components. The reason stems from random initialisation of the model's parameters (mean, covariance, and mixing weights) at the beginning of the EM algorithm. It converges to the local maxima of the log-likelihood function rather than guaranteeing a global optimum, meaning the final output is reliant on the starting position in the parameter space.

Analysis reveals critical differences between runs. F1 values range from ≈290-390 Hz in the first run compared to ≈440-650 Hz in the second run. Similarly, F2 shows more variation, ranging from ≈2270-3000 Hz in the first run versus ≈2550- 3100 Hz in the second. Also, the Gaussian components represented by the different colours on the plot occupy different positions during each run, further insinuating that the algorithm fails to assign the same mixture model to the same region in the feature space. The covariances also differ, resulting in ellipses with different orientations and eccentricities in the F1-F2 plane.

This sensitivity due to the initialisation problem is a key limitation inside the EM algorithm and suggests that multiple random restarts would be desirable to identify a solution with the highest likelihood. Despite this inconsistency, both models achieve comparable log-likelihood values after converging. Yet the main challenge remains that likelihood is not sufficient for determining the optimal model in this instance.

Q5: Binary Classification

The trained Mixture of Gaussian models from Questions 2 and 3 were used to build a binary classifier, discriminating between phoneme classes 1 and 2. Maximum Likelihood was used as the statistical method which assigned class labels based on the two density models.

For each test sample, the probability density was evaluated under both trained models p(x; θ_1) for phoneme 1 and p(x;θ_2) for phoneme 2, where θ is the parameter set mean, covariance, and mixing weights. The classifier assigns to phoneme 1 if p(x;θ_1) ¿ p(x;θ_2), else classify into phoneme 2. For a Mixture of Gaussians with k=3, the probability density is computed as:

$$p(x;\theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x;\mu_k,\Sigma_k)$$

where π_k are mixing weights, μ_k are means, and Σ_k is covariance.

The implementation (See Appendix C, Figure 4) evaluates densities for each point and compares them to determine the class label. The classifier's accuracy was calculated by comparing predicted labels against true phoneme labels in the test set. The classifier achieved 0.95 accuracy, demonstrating that the GMM successfully captures the characteristics of the two phoneme classes in the formant space. This high accuracy suggests that the three-component mixture provides good representational capacity to model the class variation while maintaining clear discrimination between the two phoneme distributions.

Q6: Classification Performance (K=6)

The classification task from Question 5 was repeated but using Mixture of Gaussian models with k=6 per phoneme class. The classifier achieved 0.96, showing a minor improvement over k=3 configuration.

The minimal gain suggests that while additional Gaussian components provide greater representation, the k=3 model captures the main distribution of the phoneme classes. Limited improvements also imply diminishing returns from increased model complexity and higher chances of overfitting. Overall, k=3 model provides a better balance between performance and complexity in this classification problem.

Q7: Classification Matrix

Decision boundaries were created by making a grid spanning the range of F1 and F2 features, then classifying each grid point using the trained models. Figures 6 and 7 (See Appendix D) display the matrices for both K=3 and K=6 configurations, with colour indicating the predicted phoneme class at each location in the F1-F2 plane.

The k=3 boundary shows a smooth separation between the two phoneme regions, reflecting the simpler model structure using three Gaussian components per class. The k=6 boundary shows more complex separation between the classes that adapts more to the variations in the training data distribution. The k=6 model creates more intricate boundaries where the two phonemes overlap, to separate clusters more precisely than k=3, capturing the intricacies of the feature space. Both models learned meaningful decision rules based on the frequency characteristics of each phoneme class.

Q8: Singularity Issue

A new dataset was created with three features: F1, F2, and F1+F2. Attempting to fit a Gaussian Mixture Model to this augmented feature space resulted in numerical errors during training (see Appendix E).

The error message indicates numerical overflow during the EM algorithm's parameter updates. This may have occurred because the third feature F1+F2 is a linear combination of the first two features, creating a rank-deficient covariance matrix. When the algorithm attempts to invert this singular covariance matrix for probability density calculations, the numerical computations become unstable and produce infinite or excessively large values. This is a fundamental limitation when working with linearly dependent features, as the covariance matrix cannot be properly inverted.

Q9: Regularisation Solution

The issue was addressed through covariance regularisation by adding a small diagonal matrix to the sample. This technique ensures the matrix remains positive and invertible by adding a small constant to the diagonal elements.

The regularised implementation successfully allowed the training of the three-dimensional space without encountering any errors. Hence, the training output shows stable convergence of the EM algorithm across all iterations. By adding minimal variance along all dimensions, we prevent the covariance matrix from becoming singular while having a negligible impact on the models' learnt parameters.

Bibliography:

Peterson, G.E. and Barney, H.L. (1951). Control Methods Used in a Study of the Vowels. The Journal of the Acoustical Society of America,

Klautau, A. (2002) Classification of Peterson & Barney's vowels using Weka.

A Appendix (Q1): Vowel Distribution

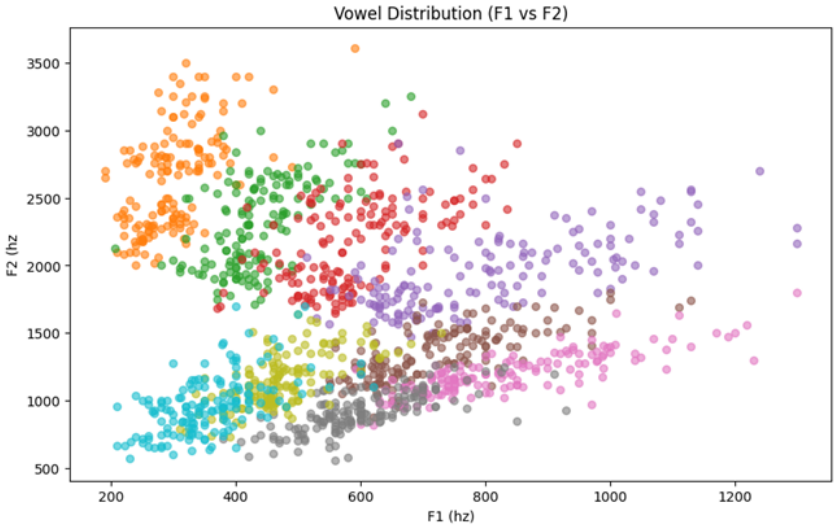


Figure 1: F1 vs F2 scatter plot for all 10 vowel classes

B Appendix (Q2): Multiple Training Runs

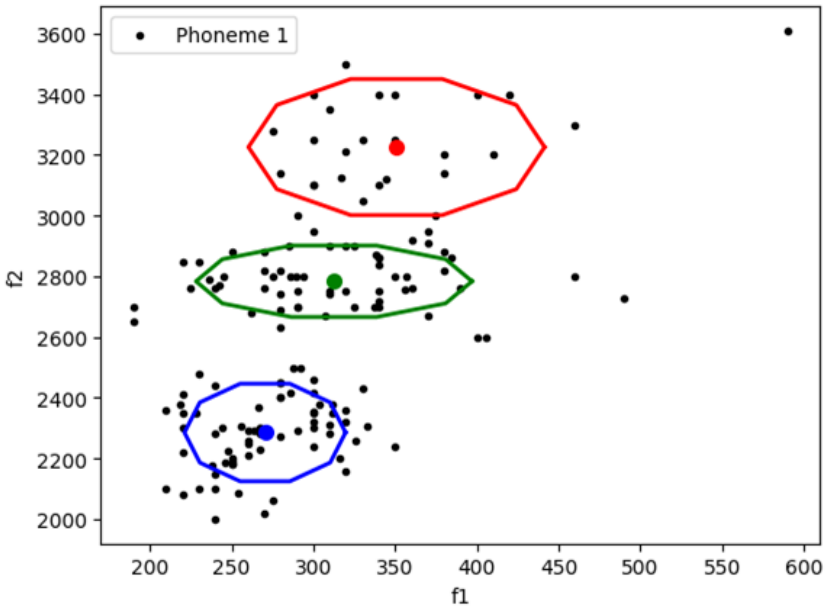


Figure 2: First MoG train with K=3 components for phoneme 1

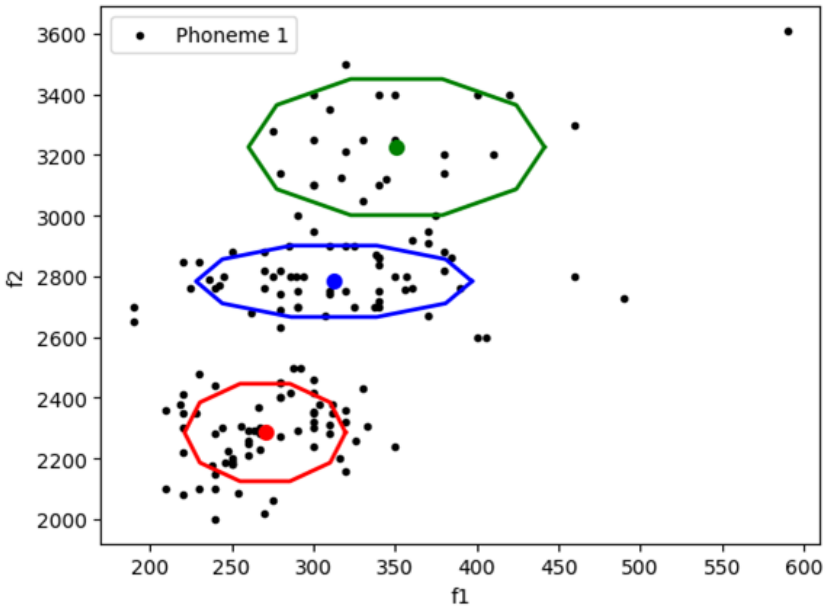


Figure 3: Second MoG train with K=3 components for phoneme 1

C Appendix (Q5): Binary Classifier, K=3

```
# Get predictions on both phonemes 1 and 2, from a GMM with k components, pretrained on phoneme 2
p_id = 2
###your code here
npy_filename = 'data/GMM_params_phoneme_{:02}_k_{:02}.npy'.format(p_id, k)
GMM_parameters_phoneme_2 = np.load(npy_filename, allow_pickle=True)
GMM_parameters_phoneme_2 = GMM_parameters_phoneme_2.item()
mu_2 = GMM_parameters_phoneme_2['mu']
s_2 = GMM_parameters_phoneme_2['s']
p_2 = GMM_parameters_phoneme_2['p']

Z_phoneme_2 = get_predictions(mu_2, s_2, p_2, X)

# classify each point using Maximum Likelihood
likelihood_1 = np.sum(Z_phoneme_1, axis=1)
likelihood_2 = np.sum(Z_phoneme_2, axis=1)
pred = np.where(likelihood_1 > likelihood_2, 1, 2)



### your code here
ground_truth = np.concatenate([np.ones(len(inds_1)), 2*np.ones(len(inds_2))])
accuracy = np.mean(pred == ground_truth)
error = 1 - accuracy
print(f"Accuracy: {accuracy:.2f}")
```

Figure 4: Implementation code for binary classification using K=3 GMMs

D Appendix (Q7): Decision Boundary Visualization

```
#loading phoneme 1 GMM
p_id = 1
npy_filename = 'data/GMM_params_phoneme_{:02}_k_{:02}.npy'.format(p_id, k)
GMM_parameters_phoneme_1 = np.load(npy_filename, allow_pickle=True)
GMM_parameters_phoneme_1 = np.ndarray.tolist(GMM_parameters_phoneme_1)
mu = GMM_parameters_phoneme_1['mu']
s = GMM_parameters_phoneme_1['s']
p = GMM_parameters_phoneme_1['p']

#loading phoneme 2 GMM
p_id = 2
npy_filename = 'data/GMM_params_phoneme_{:02}_k_{:02}.npy'.format(p_id, k)
GMM_parameters_phoneme_2 = np.load(npy_filename, allow_pickle=True)
GMM_parameters_phoneme_2 = GMM_parameters_phoneme_2.item()
mu_2 = GMM_parameters_phoneme_2['mu']
s_2 = GMM_parameters_phoneme_2['s']
p_2 = GMM_parameters_phoneme_2['p']

#creating grid points - vectors
f1_vals = np.arange(min_f1, max_f1+1)
f2_vals = np.arange(min_f2, max_f2+1)
f1_grid, f2_grid = np.meshgrid(f1_vals, f2_vals)

#flattening all points
grid_points = np.column_stack((f1_grid.flatten(), f2_grid.flatten()))

#get predictions
Z_1 = get_predictions(mu, s, p, grid_points)
Z_2 = get_predictions(mu_2, s_2, p_2, grid_points)

#compare likelihoods
likelihood_1 = np.sum(Z_1, axis=1)
likelihood_2 = np.sum(Z_2, axis=1)

#classify and reshape to grid shape
M = (likelihood_2 >= likelihood_1).astype(float).reshape(f2_grid.shape)
```

Figure 5: Implementation code for decision boundary grid generation

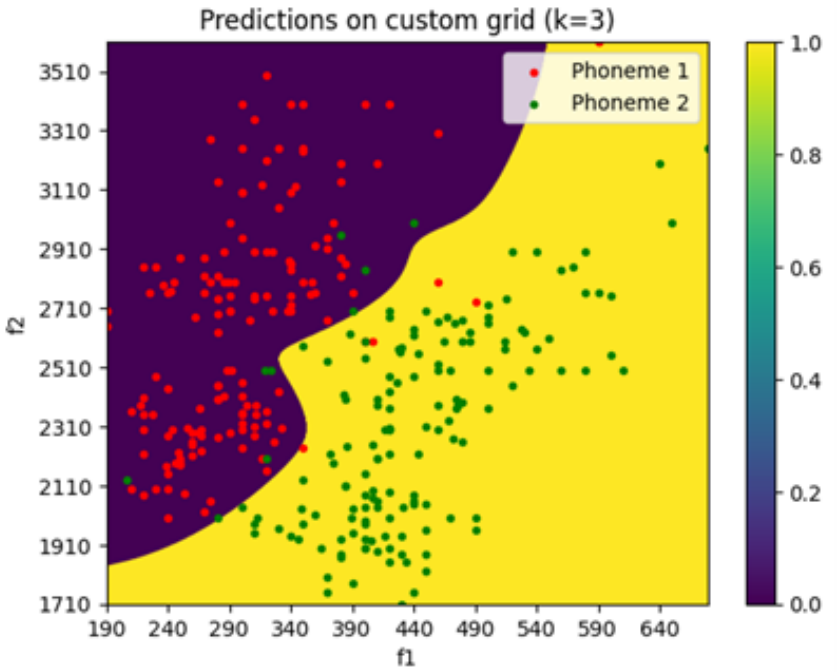


Figure 6: Decision boundary visualisation for K=3 components

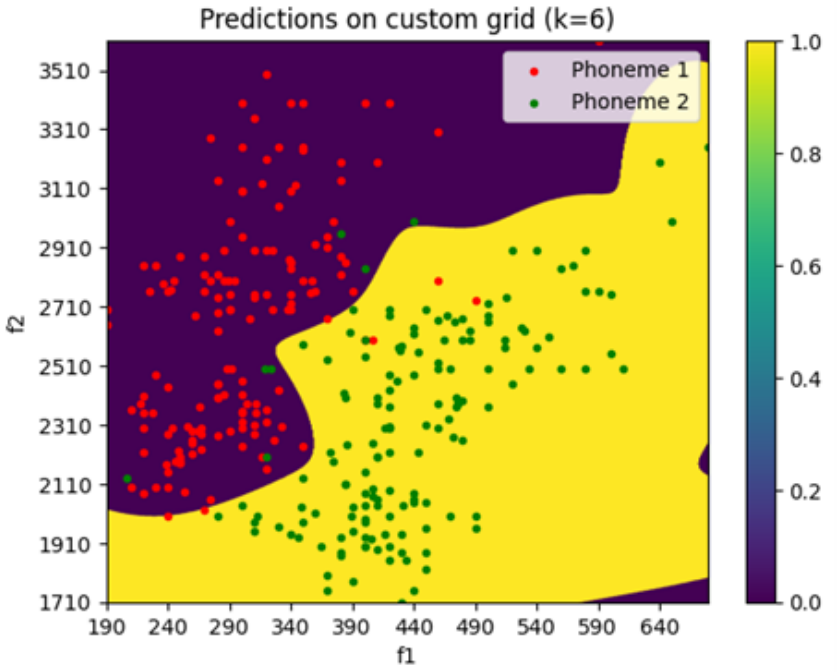


Figure 7: Decision boundary visualisation for K=6 components

E Appendix (Q8): Singularity Problem

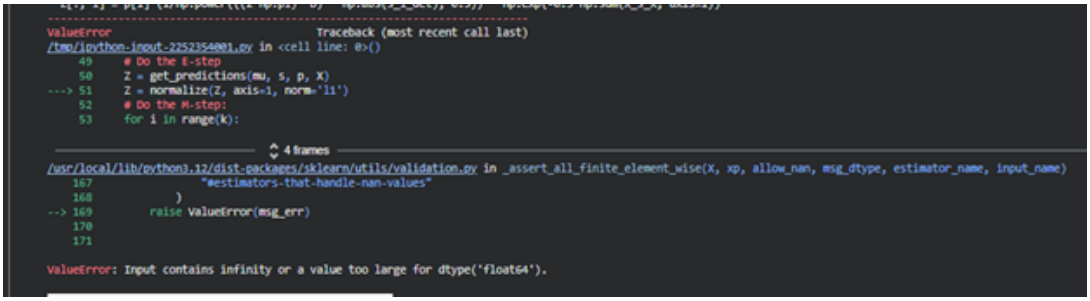


Figure 8: Error messages showing singularity issue

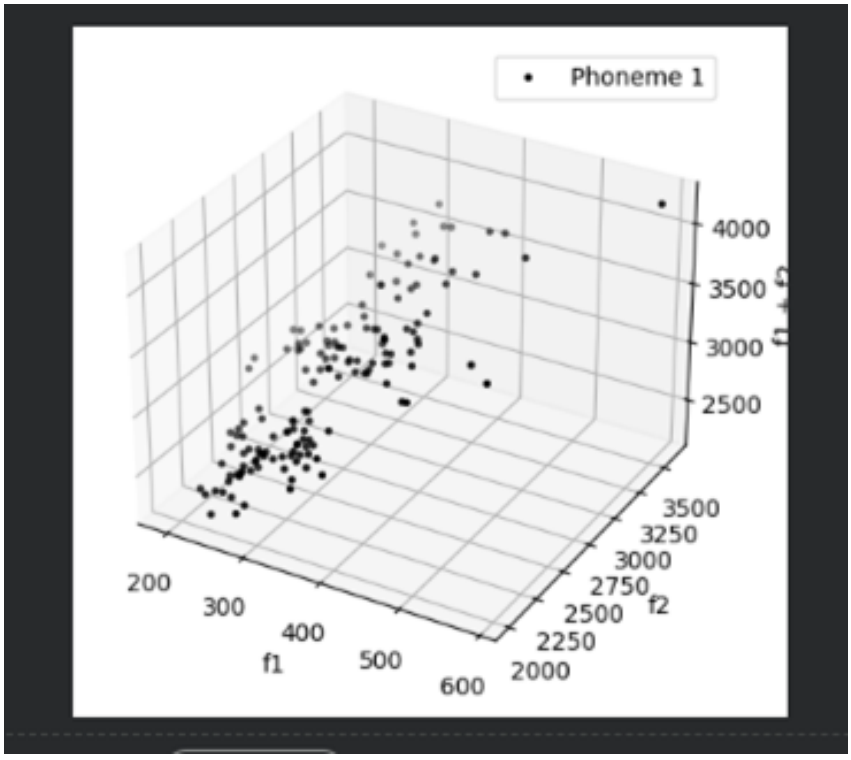


Figure 9: Failed 3D visualisation due to singularity

F Appendix (Q9): Regularisation Solution

```
# initialize covariances
for i in range(k):
    cov_matrix = np.cov(X.transpose())
    s[i,:,:] = cov_matrix/k + epilson * np.eye(D) #adding np.eye - adds values to diagonal to mitigate singularity issue
```

Figure 10: Regularisation implementation

```
# initialize covariances
for i in range(k):
    cov_matrix = np.cov(X.transpose())
    s[i,:,:] = cov_matrix/k + epilson * np.eye(D) #adding np.eye - adds values to diagonal to mitigate singularity issue
```

Figure 11: Regularisation implementation - M-step

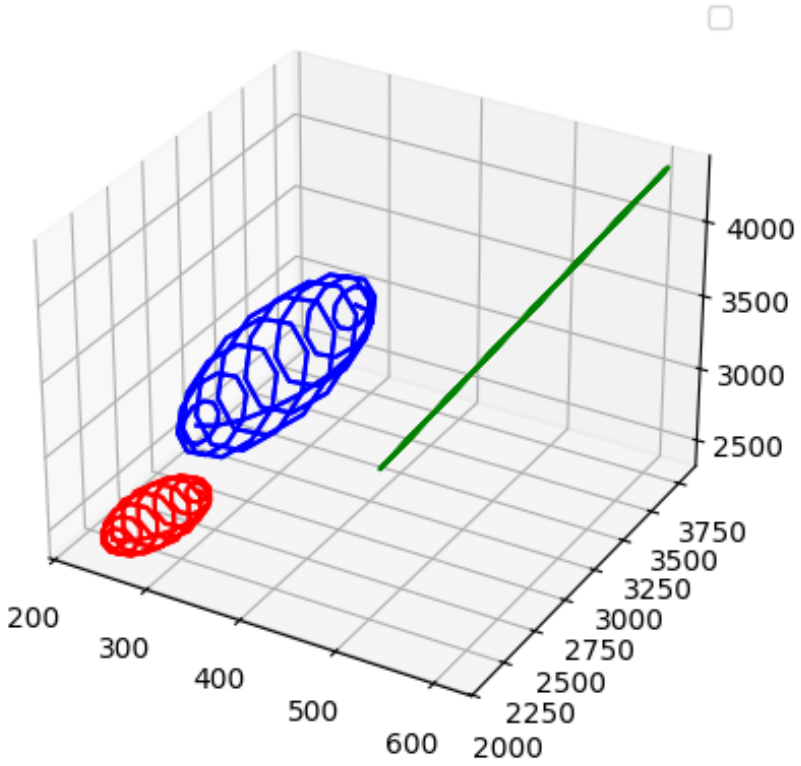


Figure 12: Successful 3D training after applying regularisation